

My final project started with the working title of “Meal Planner”, with the stated goal of uploading recipes, comparing costs, and searching based on criteria. Presently, it does not have the ability to upload recipes, as I have been using GitHub to store the .Json files for recipes, tags, etc. & GitHub does not support a method of pushing to a repository from JavaScript (for good reason).

My target audience was individuals looking to keep track of and sort their recipes, which I fit into myself. The initial plan was to store files locally, but I ran into difficulties with saving, loading, and modifying stored that way and pivoted to using a public repository on GitHub. This pivot presented its own set of challenges however, as for security reasons GitHub requires SSH to push to repositories when not using the actual website (with good reason). As such, without the ability to upload recipes directly from the website, a core functionality was lost. What remained possible is an index of recipes in a public repository, which the main page can search through and utilize a secondary page to view. With an actual server, implementation of users, user content, and further features would become possible.

HTML was used to create the elements of the web, with headers being used before sections to describe the contents, such as “Used Filters” & “Search Results”. JavaScript was responsible for the dynamic elements of the website, such as actually fetching the data from the repository, displaying search results, removing and re-adding filter elements, etc. Lastly, CSS was used to display the elements in a readable manner, rather than simply having a skeleton or bare frame. In addition CSS was used to indicate when filter elements were being hovered over, in addition to the clickable portion of the search results.

Modelling after most search options, the search box is at the top of the webpage on the main page, followed by the filters section, and lastly the results. To have the website be less bland, an off white color was used, and buttons were colored. Similar design choices were used on the recipe page, with the same background color and use of borders to separate different pieces of data on the page.

The search function relies on a JavaScript fetch to the index file, which contains information about the recipe such as recipe name, cost per serving, and applicable tags. After receiving the index, the search function checks to see if any of the recipe names in the index contain the search parameter, and displays the result if so. CSS is used to help ensure that the website content is viewable regardless of the device used.

When clicked on, the green button on the result item links to the recipe page with a url Argument “rName”, which provides the name of the recipe which will need to be fetched, from there data from the Json file is used to dynamically create the form elements.

Overall, this project taught me a lot about what goes into actually goes into building and designing functioning and non-static pages. As someone who has primarily worked with programming languages that do not have or need a visual component beyond displayed text, this project has made me much more appreciative of the CSS that the webpages I view use, and I have found myself thinking about the structure and design choices of the webpages that I view. While I doubt my focus will change much from the more back-end, this course and project have given me the mindset and tools to move from relying on solutions such as canva to challenging myself and further building my skills.